Computer Graphics and Man-Machine Interface

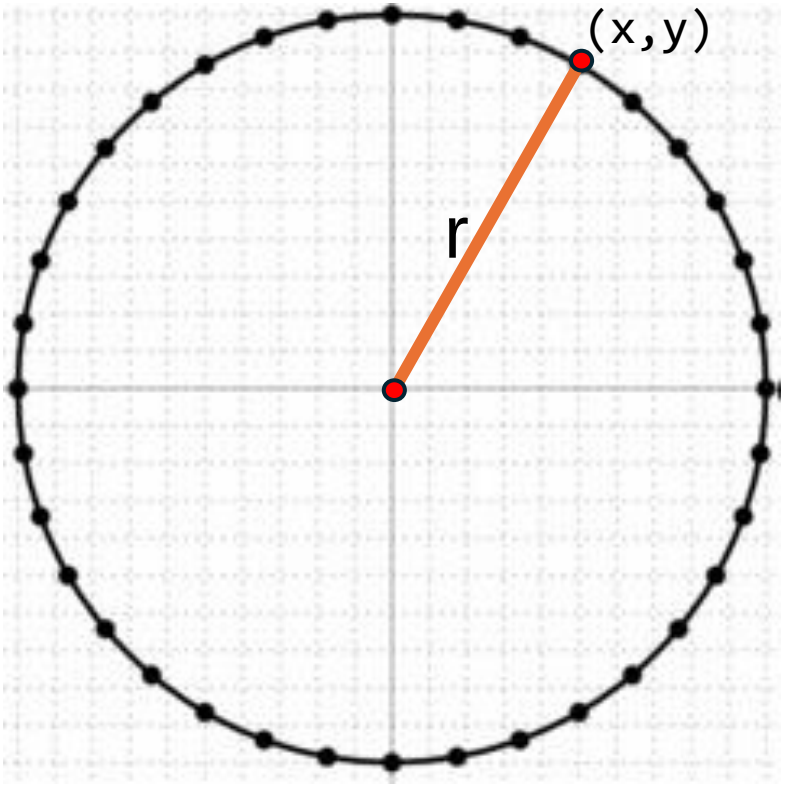
Dr. Manar Donia

CMPn322 - CMPN322 - CMP 422

Drawing circle

Drawing circle

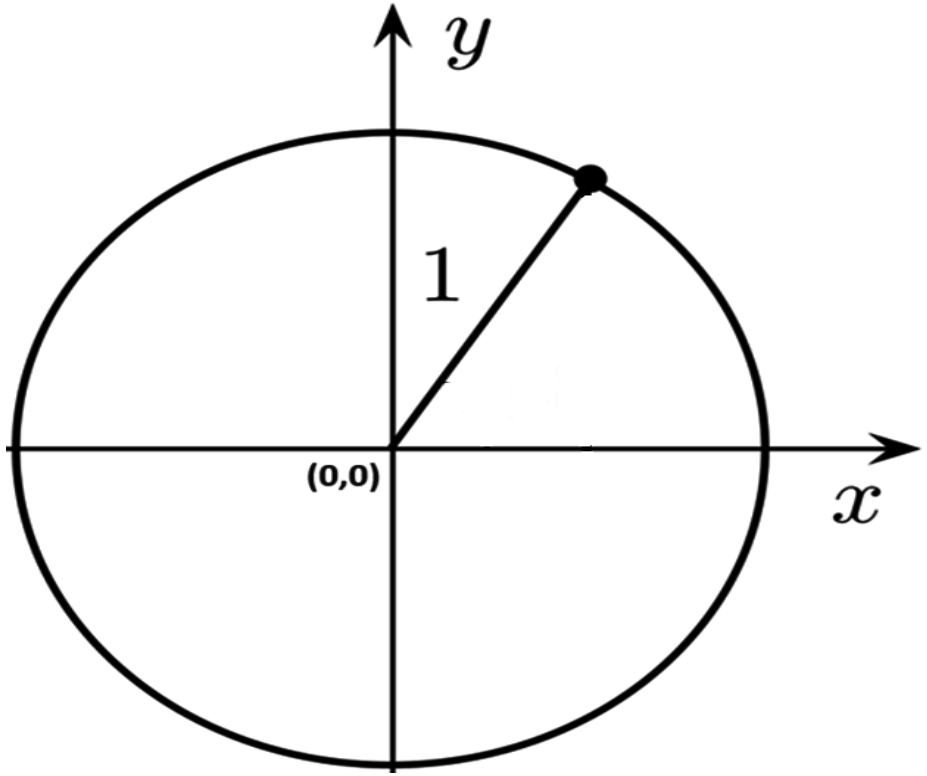
- A circle is a set of all points in a plane that are all an equal distance from a single point, the center.
- The distance from a circle's center to a point on the circle is called the radius of the circle.
- A radius is a line segment with one endpoint at the center of the circle and the other endpoint on the circle.



`glVertex2f(x,y)`

Drawing circle

- Unit circle
  - a **unit circle** is the circle of radius 1 centered at the origin (0, 0) in the Cartesian coordinate system.

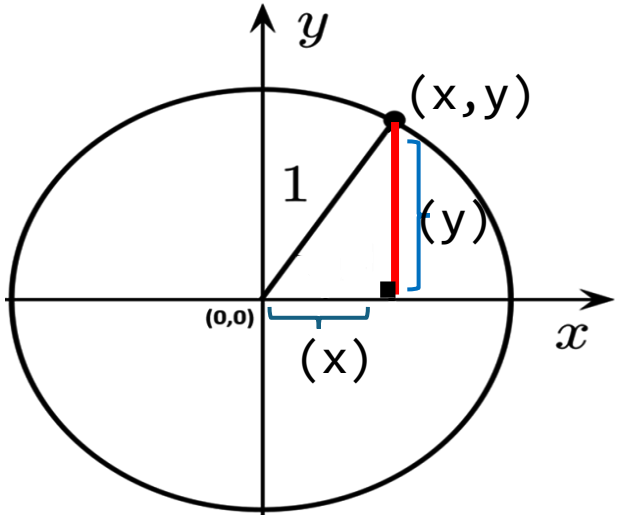


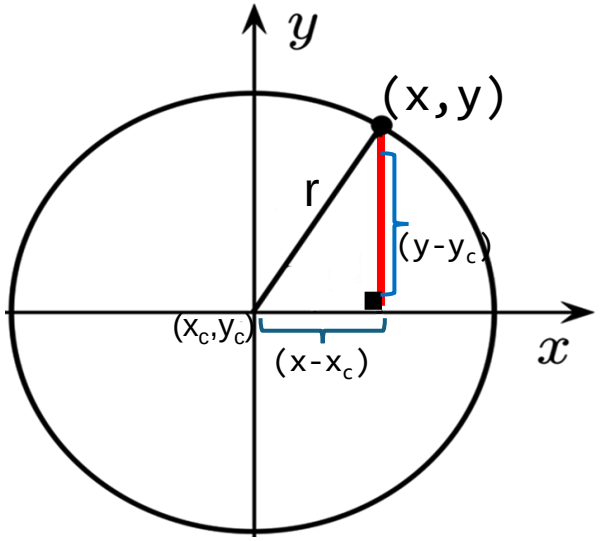
Drawing circle

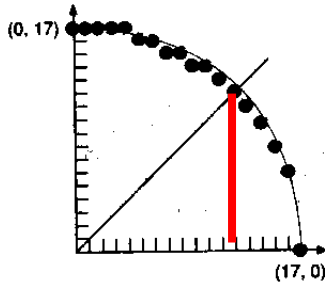
- Unit circle
  - a **unit circle** is the circle of radius 1 centered at the origin (0, 0) in the Cartesian coordinate system.
- The **Pythagoras theorem** explains the relationship between the three sides of a right-angled triangle.
  - the square of the hypotenuse is equal to the sum of the squares of the other two sides of a triangle.
  - $x^2 + y^2 = h^2$→ h=r
  - Since its unit circle so r=1
    - $x^2 + y^2 = 1$
    - $y = \sqrt{1 - x^2}$
- We could draw a quarter circle by incrementing x from 0 to R in unit steps and solving for +y for each step.
- For another circle rather than unit circle

$$y = y_c + \sqrt{r^2 - (x - x_c)^2}$$

Method needs lots of computation, and gives non-uniform pixel spacing



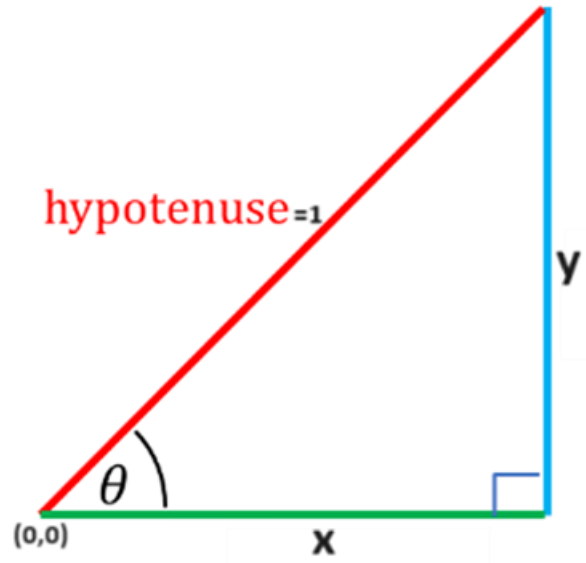




Drawing circle

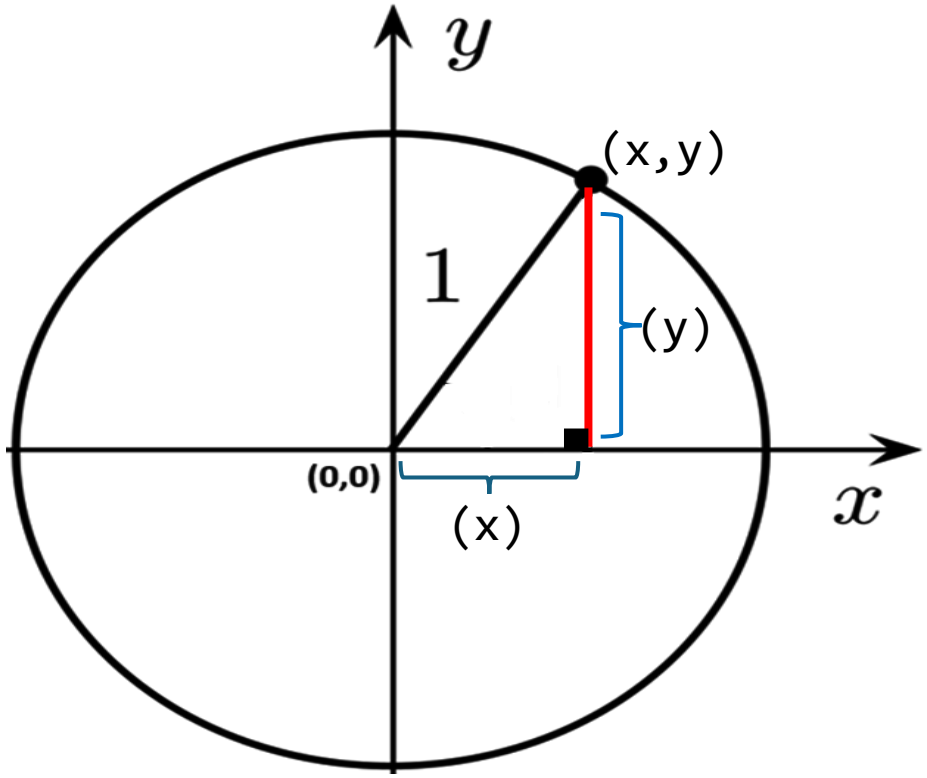
- Trigonometry in right-angled triangles:
  - In a right-angled triangle:
    - The ratio of the opposite side to the hypotenuse is the sin of angle θ.
      - $\sin\theta = \frac{y}{r}$
    - The ratio of the adjacent side to the hypotenuse is the cos of angle θ.
      - $\cos\theta = \frac{x}{r}$

$x = r\cos\theta$ and $y = \mathrm{r}\sin\theta$



Drawing circle

- Unit circle
- Since r=1
  - $x = \cos\theta$
  - $y = \sin\theta$



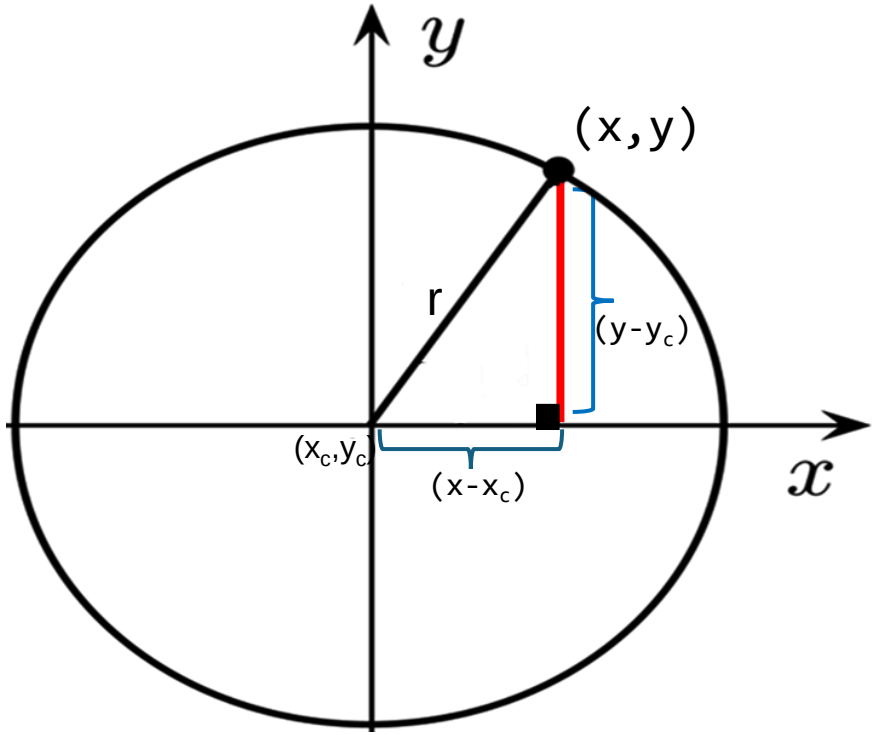
Drawing circle

- Another circle
  - $\sin\theta = \frac{y}{r}$
    - $y = y - y_c$
  - $\sin\theta = \frac{y - y_c}{r}$

$$y = y_c + \sin\theta * r$$

  - $\cos\theta = \frac{x}{r}$
    - $x = x - x_c$
  - $\cos\theta = \frac{x - x_c}{r}$

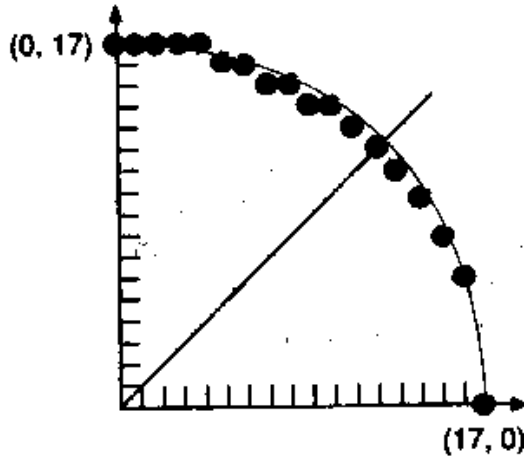
$$x = x_c + \cos\theta * r$$



Midpoint circle algorithm
Scan Converting Circles

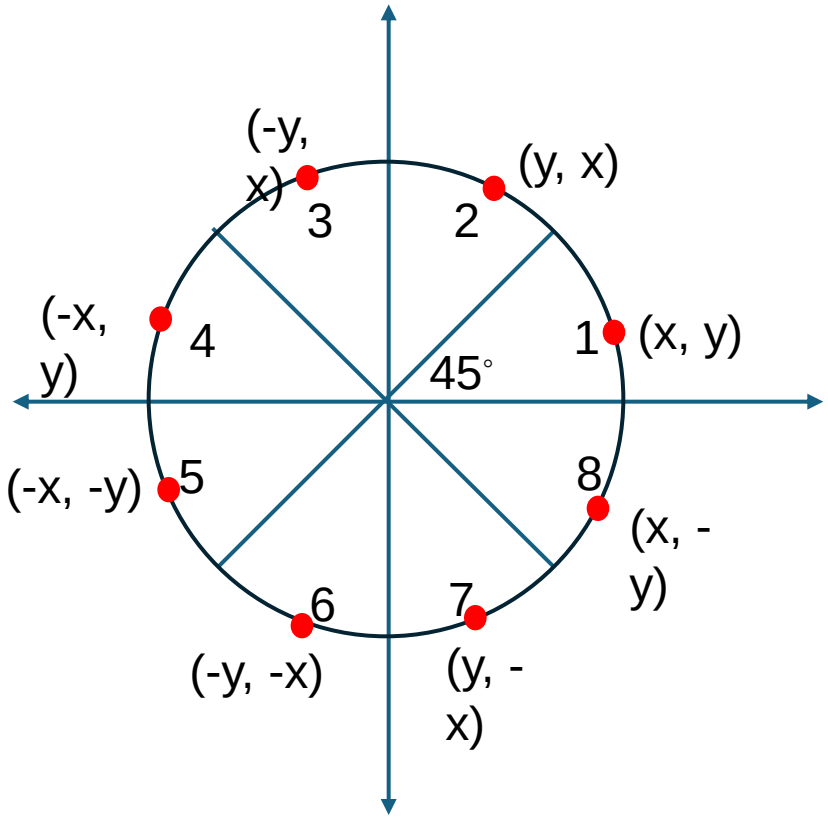
- $R^2 = x^2+y^2$
- $x^2+y^2-R^2 = 0$
- Decision parameter(P)= $x^2+y^2-R^2$

If P= 0 then it is on the circle.
P> 0 then it is outside the circle.
P< 0 then it is inside the circle.



Try to adapt the Bresenham midpoint approach

Drawing circle

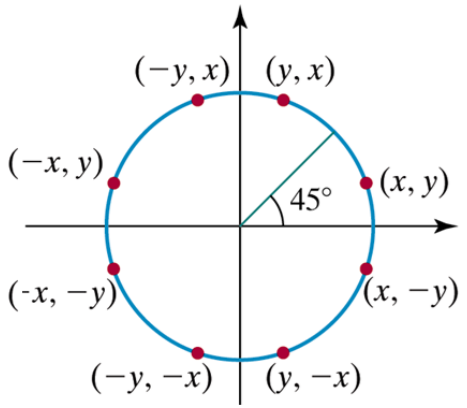


Symmetry of a circle. Calculation of a circle point(x, y) in one octant yields the circle points shown for other seven octants.

Midpoint Circle Algorithm

1. Input radius r and circle center (x_c, y_c), and obtain the first point on the circumference of a circle centered on the origin as

$$(x_0, y_0) = (0, r)$$



2. Calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

3. At each x_k position, starting at $k = 0$, perform the following test: If $p_k < 0$, the next point along the circle centered on (0, 0) is (x_{k+1}, y_k) and

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

   Otherwise, the next point along the circle is $(x_k + 1, y_k - 1)$ and

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

   where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$.

4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values:

$$x = x + x_c, \qquad y = y + y_c$$

6. Repeat steps 3 through 5 until $x \geq y$.

Midpoint Circle Drawing

Given a circle radius $r = 10$, we demonstrate the midpoint circle algorithm by determining positions along the circle octant in the first quadrant from $x = 0$ to $x = y$.

Initial Point (x0,y0)→(0,10)

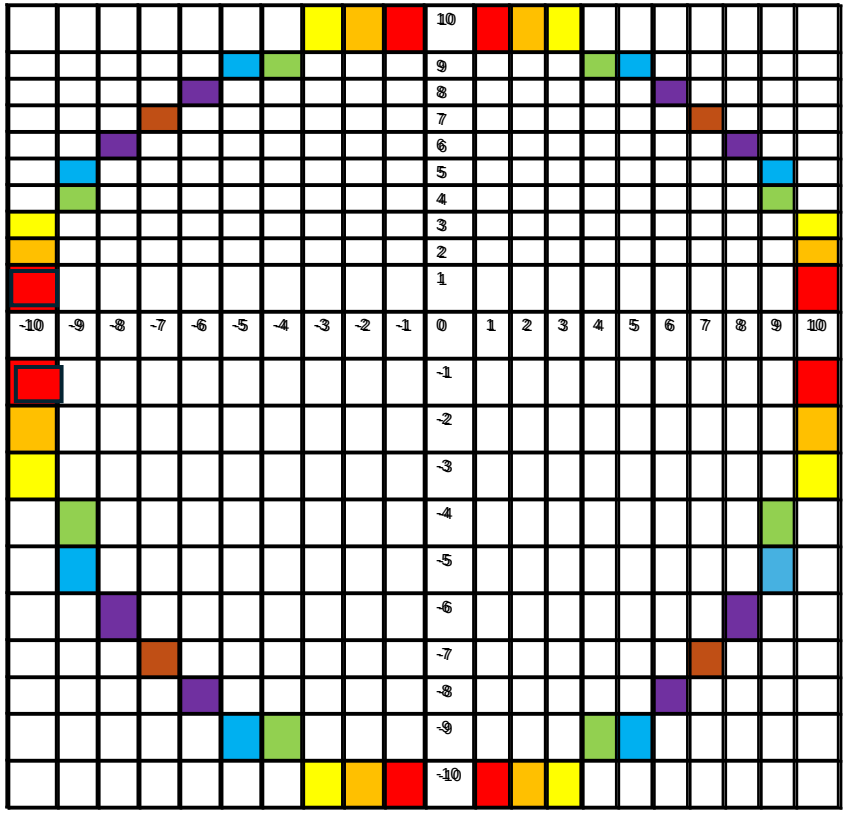
if p_k <0 the midpoint is inside the circle	if p_k >0 the midpoint is outside the circle
• $y_{k+1} = y_k$ • $P_{k+1}=p_K+2x_{k+1}+1$	• $y_{k+1} = y_k-1$ • $P_{k+1}=p_K+2x_{k+1}+1-2y_{K+1}$

Repeat until $x \geq y$.

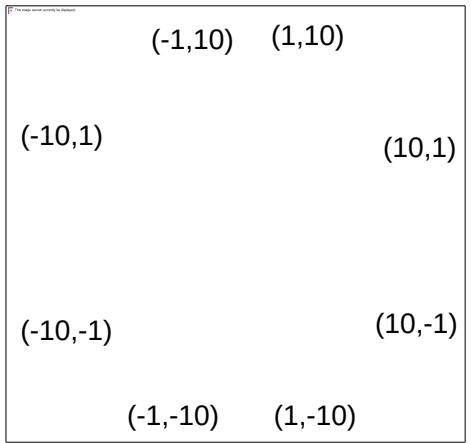
P	X	Y	$2x_{k+1}$	$2y_{k+1}$
-	0	10	-	-
-9	**1**	10	2	20
-6	**2**	10	4	20
-1	**3**	10	6	20
6	**4**	9	8	18
-3	**5**	9	10	18
8	**6**	8	12	16
5	**7**	7	-	-

Bresenham Circle with center (0, 0) and radius 10 - second octant

k	P_k	(X_{k+1}, Y_{k+1})	$2X_{k+1}$	$2Y_{k+1}$
0	-9	(1, 10)	2	20
1	-6	(2, 10)	4	20
2	-1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	-3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14



Eight-way Symmetry
- only one octant's calculation needed

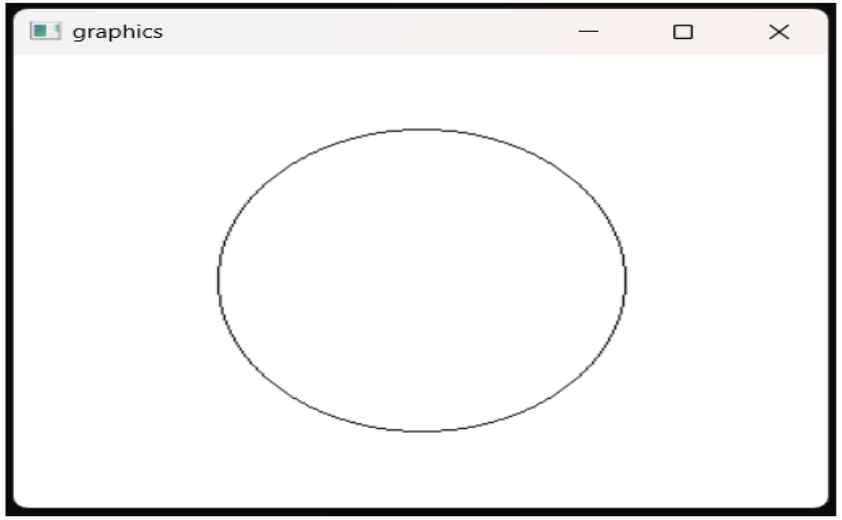


Setpixel(x,y)➔ Setpixel(1,10)
Setpixel(y,x)➔ Setpixel(10,1)
Setpixel(y,-x)➔ Setpixel(10,-1)
Setpixel(x,-y)➔ Setpixel(1,-10)
Setpixel(-x,-y)➔ Setpixel(-1,-10)
Setpixel(-y,-x)➔ Setpixel(-10,-1)
Setpixel(-y,x)➔ Setpixel(-10,1)
Setpixel(-x,y)➔ Setpixel(-1,10)

Setpixel(x,y)➔ Setpixel(xc+x, yc+y)

Midpoint circle algorithm

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if it doesn't written there will be no display
    glPointSize(1);
    circleMidpoint(0, 0, 100,GL_POINTS);
    glFlush();
}
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
        x++;
        if (p0 < 0)
            p0 = p0 + 2 * x + 1;
        else
        {
            y--;
            p0 = p0 + 2 * x -2*y + 1;
        }
        circleplot(xc, yc, x, y, mode);
    }
}
void circleplot(int xc, int yc, int x, int y, GLenum mode)
{
    glBegin(mode);
    glVertex2f(xc + x, yc + y);
    glVertex2f(xc - x, yc + y);
    glVertex2f(xc + x, yc - y);
    glVertex2f(xc - x, yc - y);
    glVertex2f(xc + y, yc + x);
    glVertex2f(xc - y, yc + x);
    glVertex2f(xc + y, yc - x);
    glVertex2f(xc - y, yc - x);
    glEnd();
}
```



Ellipse-Generating Algorithms

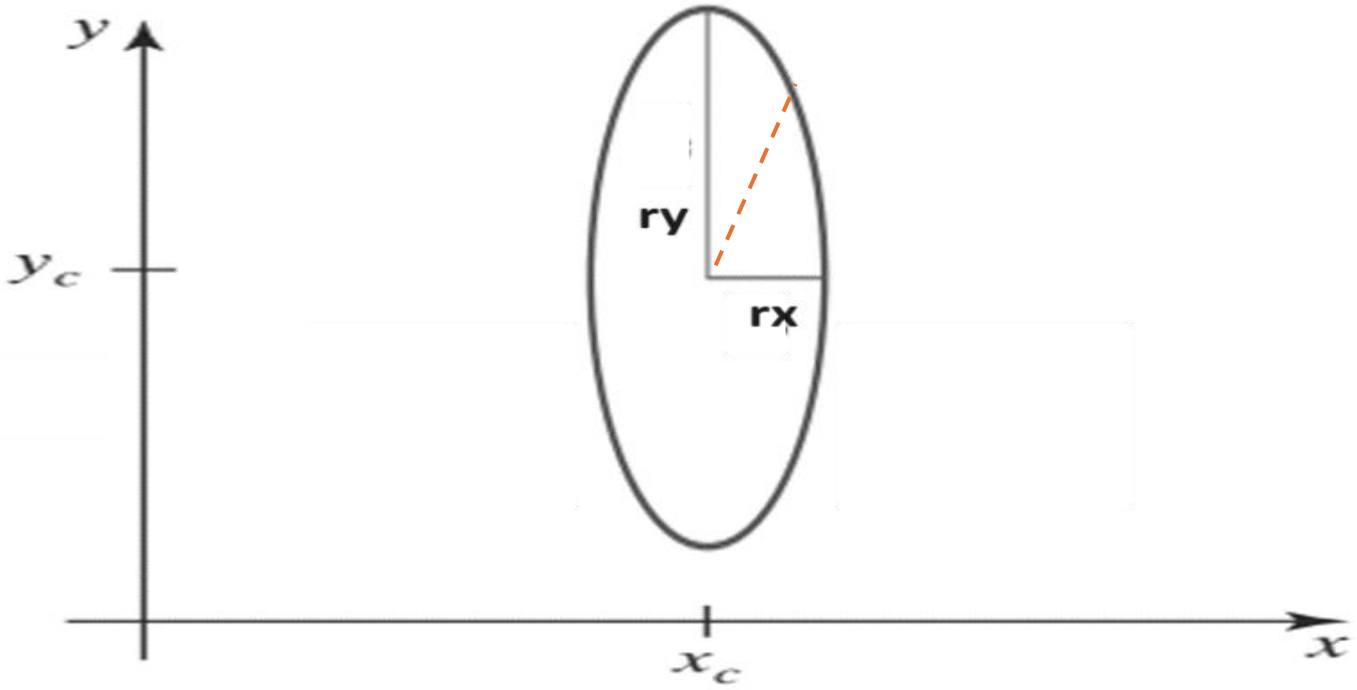


FIGURE 17
Ellipse centered at (x_c, y_c) with semimajor axis r_y and semiminor axis r_x.

Symmetry of a circle. Calculation of a circle point (x, y) in one octant yields the circle points shown for the other seven octants.

Ellipse-Generating Algorithms

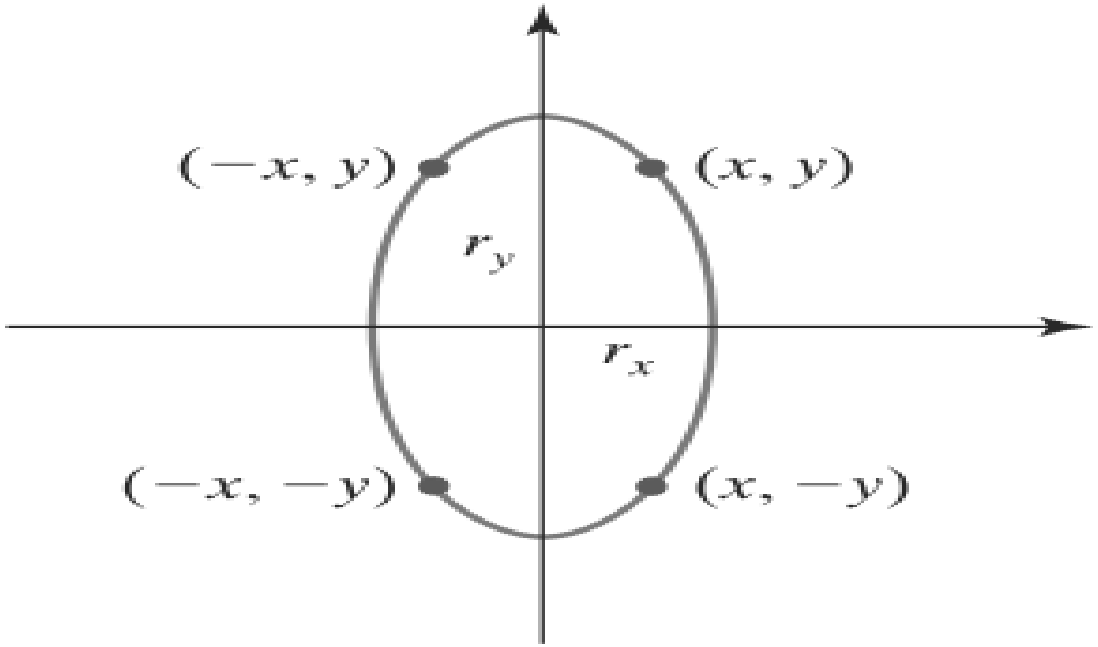
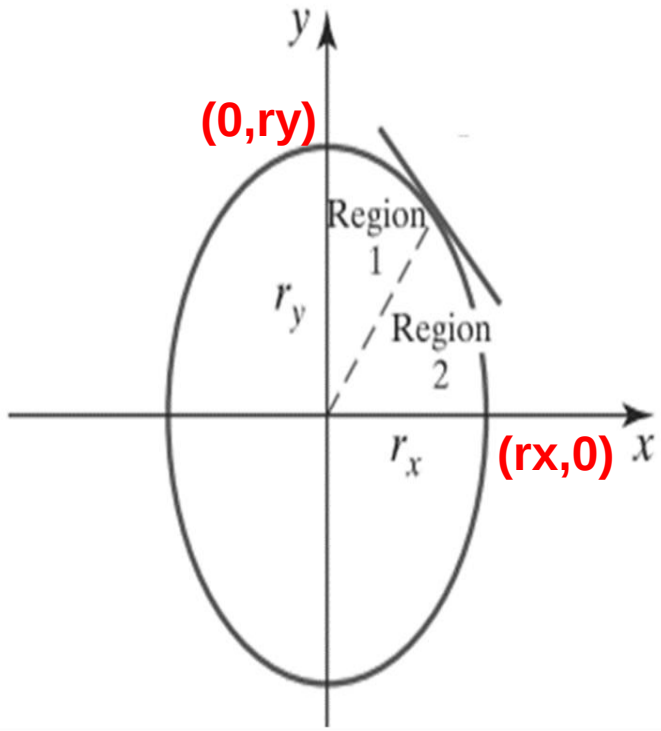
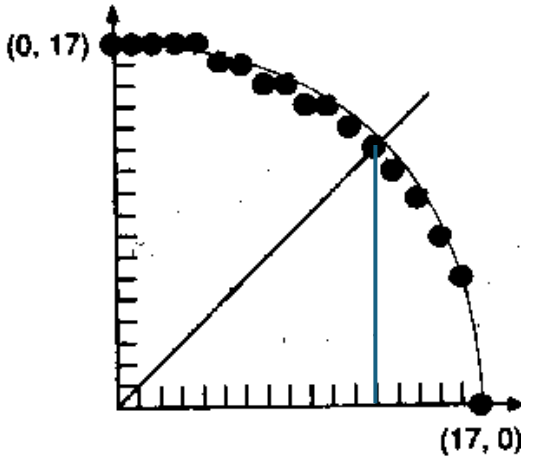


FIGURE 19
Symmetry of an ellipse. Calculation of a point (x, y) in one quadrant yields the ellipse points shown for the other three quadrants.

Ellipse-Generating Algorithms



Ellipse processing regions. Over region 1, the magnitude of the ellipse slope is less than 1.0; over region 2, the magnitude of the slope is greater than 1.0.



Region 1 increment x , calculate y
Region 2 decrement y, calculate x

Midpoint Ellipse Algorithm

1. Input r_x, r_y, and ellipse center (x_c, y_c), and obtain the first point on an ellipse centered on the origin as

$$(x_0, y_0) = (0, r_y)$$

2. Calculate the initial value of the decision parameter in region 1 as

$$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$$

3. At each x_k position in region 1, starting at $k = 0$, perform the following test: If $p1_k < 0$, the next point along the ellipse centered on $(0, 0)$ is (x_{k+1}, y_k) and

$$p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2$$

Otherwise, the next point along the ellipse is $(x_k + 1, y_k - 1)$ and

$$p1_{k+1} = p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$$

with

$$2r_y^2 x_{k+1} = 2r_y^2 x_k + 2r_y^2, \qquad 2r_x^2 y_{k+1} = 2r_x^2 y_k - 2r_x^2$$

and continue until $2r_y^2 x \geq 2r_x^2 y$.

4. Calculate the initial value of the decision parameter in region 2 as

$$p2_0 = r_y^2\left(x_0 + \frac{1}{2}\right)^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2$$

where (x_0, y_0) is the last position calculated in region 1.

5. At each y_k position in region 2, starting at $k = 0$, perform the following test: If $p2_k > 0$, the next point along the ellipse centered on $(0, 0)$ is $(x_k, y_k - 1)$ and

$$p2_{k+1} = p2_k - 2r_x^2 y_{k+1} + r_x^2$$

Otherwise, the next point along the ellipse is $(x_k + 1, y_k - 1)$ and

$$p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$$

using the same incremental calculations for x and y as in region 1. Continue until $y = 0$.

6. For both regions, determine symmetry points in the other three quadrants.

7. Move each calculated pixel position (x, y) onto the elliptical path centered on (x_c, y_c) and plot these coordinate values:

$$x = x + x_c, \qquad y = y + y_c$$

Ellipse-region1

Initial Point (x0,y0)→(0,r) &P0=$r_y^2-r_x^2r_y+1/4r_x^2$

if $p_k<0$ the midpoint is inside the ellipse	if $p_k>0$ the midpoint is outside the ellipse
$x_{k+1} = x_k + 1$ • $y_{k+1} = y_k$ • $P_{k+1} = p_K+2r_y^2x_{k+1}+r_y^2$	$x_{k+1} = x_k + 1$ • $y_{k+1} = y_k - 1$ • $P_{k+1}=p_K+2r_y^2x_{k+1}-2r_x^2y_{k+1}+r_y^2$

Repeat until $2r_y^2x \geq 2r_x^2y$.

Ellipse-region2

Initial Point (x0,y0)→ Last point of region 1 &
$P0=r_y^2(x0+1/2)^2+r_x^2(y0-1)^2-r_x^2r_y^2$

if p_k <0 the midpoint is inside the ellipse	if p_k >0 the midpoint is outside the ellipse
$X_{k+1}=x_k+1$ • $Y_{k+1}=y_k-1$ • $P_{k+1}=p_K+2r_y^2x_{k+1}-2r_x^2y_{k+1}+r_x^2$	• $X_{k+1}=x_k$ • $Y_{k+1}=y_k-1$ • $P_{k+1}=p_K-2r_x^2y_{k+1}+r_x^2$

Repeat until we reach point (rx,0)

EXAMPLE 3 Midpoint Ellipse Drawing

Given input ellipse parameters $r_x = 8$ and $r_y = 6$, we illustrate the steps in the midpoint ellipse algorithm by determining raster positions along the ellipse path in the first quadrant. Initial values and increments for the decision parameter calculations are

For region 1, the initial point for the ellipse centered on the origin is $(x_0, y_0) = (0, 6)$, and the initial decision parameter value is

$$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 = -332$$

k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	−332	(1, 6)	72	768
1	−224	(2, 6)	144	768
2	−44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	−108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

Initial Point (x0,y0)→(0,6)

if p_k<0 the midpoint is inside the ellipse

- $x_{k+1} = x_{k+1}$
- $y_{k+1} = y_k$
- $P_{k+1} = p_K + 2r_y^2 x_{k+1} + r_y^2$

if p_k>0 the midpoint is outside the ellipse

- $x_{k+1} = x_{k+1}$
- $y_{k+1} = y_{k-1}$
- $P_{k+1} = p_K + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$

Repeat until $2r_y^2 x \geq 2r_x^2 y$.

We now move out of region 1 because $2r_y^2 x > 2r_x^2 y$.

For region 2, the initial point is $(x_0, y_0) = (7, 3)$ and the initial decision parameter is **$P0=r_y^2(x0+1/2)^2+r_x^2(y0-1)^2-r_x^2r_y^2$**

$$p2_0 = f_{\text{ellipse}}\left(7 + \frac{1}{2}, 2\right) = -151$$

The remaining positions along the ellipse path in the first quadrant are then calculated as

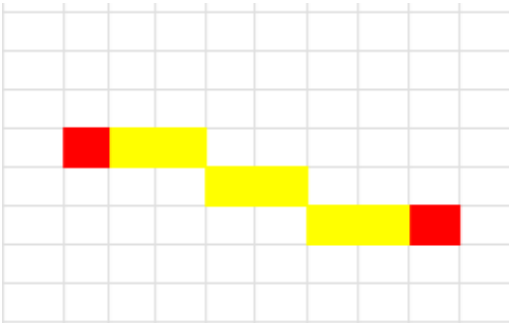
k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	−151	(8, 2)	576	256
1	233	(8, 1)	576	128
2	745	(8, 0)	—	—

Initial Point (x0,y0)→ Last point of region 1 (7,3) &

if p_k <0 the midpoint is inside the ellipse	**if p_k >0 the midpoint is outside the ellipse**
• $X_{k+1} = x_{k+1}$ • $Y_{k+1} = y_{k-1}$ • $P_{k+1} = p_K + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$	• $X_{k+1} = x_k$ • $Y_{k+1} = y_{k-1}$ • $P_{k+1} = p_K - 2r_x^2 y_{k+1} + r_x^2$

Implementation Methods for Antialiasing

- Line segments and other graphics primitives generated by the raster algorithms discussed earlier **a jagged, or stair-step, appearance**
- This distortion of information due to low-frequency sampling (under sampling) is **called aliasing**.
- We can improve the appearance of displayed raster lines by applying **antialiasing methods** that compensate for the under sampling process.
- One way to increase sampling rate with raster systems is simply to display objects at higher resolution. **However, even at the highest resolution possible with current technology, the jaggies will be apparent to some extent.**
- **There is a limit to how big we can make the frame buffer and still maintain the refresh rate at 60 frames or more per second.**



Implementation Methods for Antialiasing

- A straightforward antialiasing method is to increase sampling rate by treating the screen as if it were covered with a finer grid than is actually available.
- We can then use multiple sample points across this finer grid to determine an appropriate intensity level for each screen pixel. This technique is called super-sampling (or postfiltering, because the general method involves computing intensities at subpixel grid positions and then combining the results to obtain the pixel intensities).
- **Supersampling** algorithms are often implemented by giving more weight to subpixels near the center of a pixel area because we would expect these subpixels to be more important in determining the overall intensity of a pixel. For the 3 × 3 pixel subdivisions we have considered so far, a weighting scheme

1	2	1
2	4	2
1	2	1





- Write the implementation of an ellipse midpoint algorithm?

Thank You